Artificial Intelligence

BS (CS) _SPRING_2026

# Lab_11 Manual




## Learning Objectives:

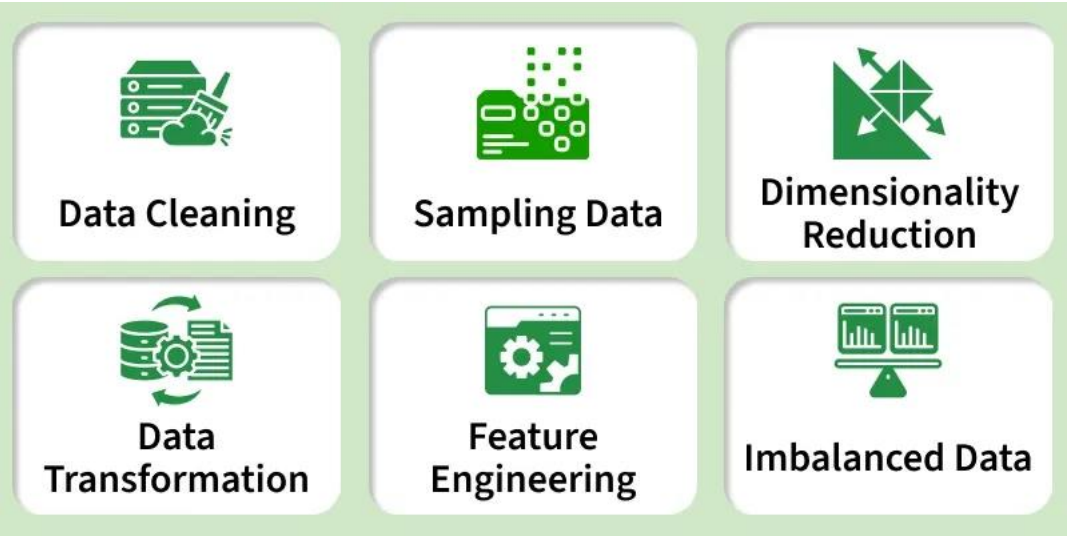
1. Data Preprocessing

# What is Data Preprocessing in Machine Learning?

**Data preprocessing** is the crucial step of transforming raw data into a clean and structured format that machine learning algorithms can work with. Raw data often contains missing values, irrelevant information, or inconsistencies that can confuse a machine learning model, leading to inaccurate results.

Preprocessing helps by preparing the data and making it suitable for analysis. This includes a range of activities such as cleaning data, filling in missing values, normalizing and scaling features, and removing duplicates or irrelevant data points. Without proper data preprocessing, even the most advanced machine learning models may struggle to provide accurate predictions.

In simple terms, data preprocessing acts as a foundation that helps machine learning models understand and learn from data more effectively. By ensuring the data is in the right shape, we help the model produce better and more reliable results.




# Why is Data Preprocessing Important?

Raw data, in its original form, often contains various issues that can negatively affect the performance of machine learning models. This makes data preprocessing a critical step before feeding data into any model. Let's look at why it's important:

### 1. Improve Data Quality

Raw data may contain errors, inconsistencies, or irrelevant information. By cleaning and correcting these issues, data preprocessing improves the overall quality of the dataset. High-quality data leads to better, more accurate models.

**2. Handle Missing Data**

In many datasets, you will find missing values, which can confuse a model. Common techniques for handling missing data include:

- **Imputation**: Filling missing values with the mean, median, or mode.
- **Deletion**: Removing rows or columns with missing values (though this should be done with caution).
- **Feature Engineering**: Creating new features to capture missing data patterns.

**3. Normalize and Scale Data**

Different machine learning algorithms, like neural networks and support vector machines, are sensitive to the scale of input features. Features with larger ranges can dominate those with smaller ranges, skewing results. Techniques like **min-max scaling** or **standardization** can normalize data to bring all features onto a similar scale.

**4. Eliminate Duplicate Records**

Duplicate records can mislead the model into thinking certain patterns are more frequent than they actually are. Identifying and removing duplicates ensures that the model learns from unique data points.

**5. Handle Outliers**

Outliers are data points that are significantly different from the rest of the data and can distort model predictions. Techniques like **capping** (limiting values within a certain range) or **removal** of outliers can help minimize their impact on the model.

By addressing these issues through preprocessing, we ensure that the data is ready for machine learning algorithms, which leads to more accurate, stable, and generalizable models.

# Key Benefits of Data Preprocessing

Effective data preprocessing can bring several key benefits to machine learning models, significantly improving their performance. Here are some of the main advantages:

**1. Enhanced Model Performance**

Preprocessed data helps machine learning models perform better by ensuring the data is clean, well-organized, and relevant. By removing noise (such as errors or irrelevant features), models can focus on learning meaningful patterns, leading to improved accuracy and generalizability.

For example, when noisy data or outliers are removed, the model is less likely to be misled by incorrect data points, making its predictions more reliable.

### 2. Reduced Training Time

Clean data reduces the amount of time the model needs to train. With fewer irrelevant features and more consistent data, the model can learn faster, which is especially important when working with large datasets. Preprocessing makes the training process more efficient by ensuring the model doesn't waste time on unnecessary or misleading data.

### 3. Improved Model Interpretability

Data preprocessing can also make it easier to understand the relationships between features and the target variable. When data is normalized, scaled, and cleaned, it becomes clearer how each feature impacts the model's predictions. This is particularly useful when you need to explain or interpret your model's results to non-technical stakeholders.

### 4. Increased Model Stability

Handling missing values, removing duplicates, and eliminating outliers all contribute to a more stable model that produces consistent results across different data samples. Stable models are crucial for making accurate predictions in real-world scenarios, where new data points might not always match perfectly with the training data.

Now that we understand the benefits, let's dive into the essential steps involved in effective data preprocessing

## Steps-by-Step implementation

Let's implement various preprocessing features,

### Step 1: Import Libraries and Load Dataset

We prepare the environment with libraries liike pandas, numpy, scikit learn, matplotlib and seaborn for data manipulation, numerical operations, visualization and scaling. Load the dataset for preprocessing.

```
import pandas as pd
import numpy as np
```

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('Geeksforgeeks/Data/diabetes.csv')
df.head()
```

**Output:**

| | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI | DiabetesPedigreeFunction | Age | Outcome |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 6 | 148 | 72 | 35 | 0 | 33.6 | 0.627 | 50 | 1 |
| 1 | 1 | 85 | 66 | 29 | 0 | 26.6 | 0.351 | 31 | 0 |
| 2 | 8 | 183 | 64 | 0 | 0 | 23.3 | 0.672 | 32 | 1 |
| 3 | 1 | 89 | 66 | 23 | 94 | 28.1 | 0.167 | 21 | 0 |
| 4 | 0 | 137 | 40 | 35 | 168 | 43.1 | 2.288 | 33 | 1 |

### Step 2: Inspect Data Structure and Check Missing Values

We understand dataset size, data types and identify any incomplete (missing) data that needs handling.

- **df.info():** Prints concise summary including count of non-null entries and data type of each column.
- **df.isnull().sum():** Returns the number of missing values per column.

```
df.info()
print(df.isnull().sum())
```

**Output:**

```
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```


```
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```


**Step 3: Statistical Summary and Visualizing Outliers**

Get numeric summaries like mean, median, min/max and detect unusual points (outliers). Outliers can skew models if not handled.

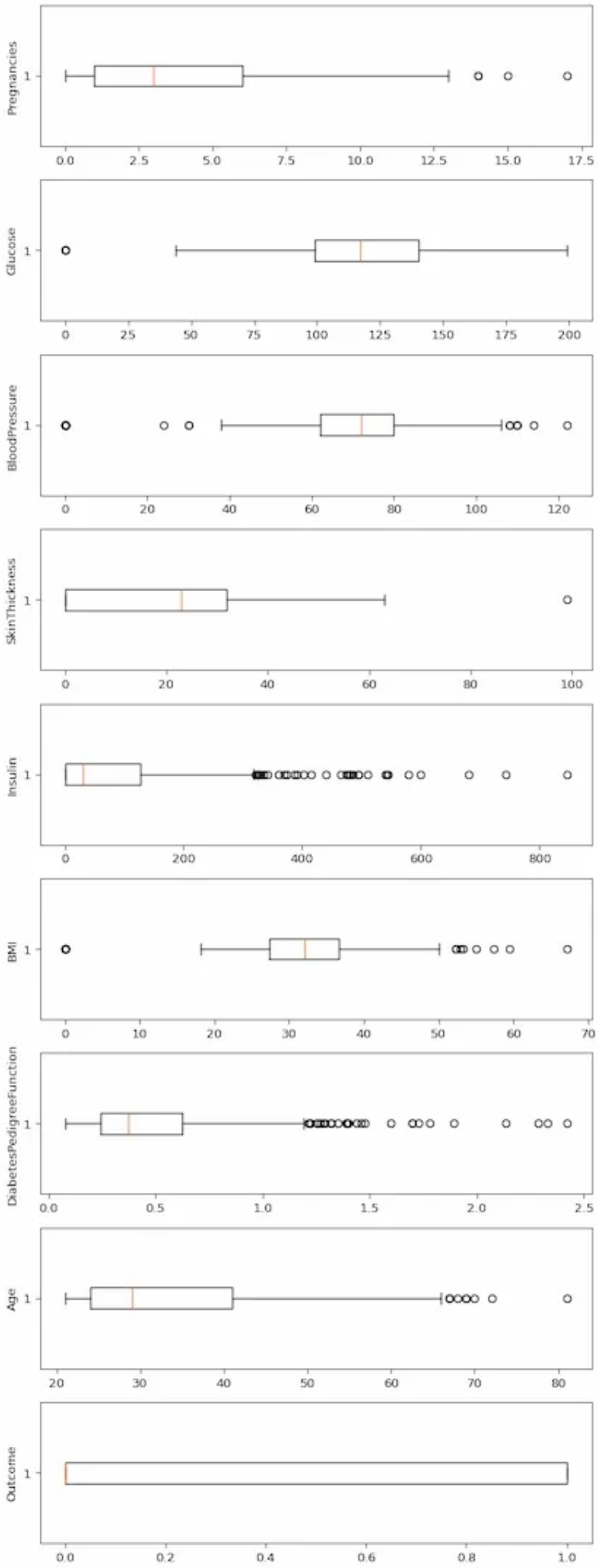
- **df.describe():** Computes count, mean, std deviation, min/max and quartiles for numerical columns.

- **Boxplots:** Visualize spread and detect outliers using matplotlib's boxplot().

```
df.describe()

fig, axs = plt.subplots(len(df.columns), 1, figsize=(7, 18),
dpi=95)
for i, col in enumerate(df.columns):
    axs[i].boxplot(df[col], vert=False)
    axs[i].set_ylabel(col)
plt.tight_layout()
plt.show()
```

**Output:**


Pregnancies
1
0.0 2.5 5.0 7.5 10.0 12.5 15.0 17.5
Glucose
1
0 25 50 75 100 125 150 175 200
BloodPressure
1
0 20 40 60 80 100 120
SkinThickness
1
0 20 40 60 80 100
Insulin
1
0 200 400 600 800
BMI
1
0 10 20 30 40 50 60 70
DiabetesPedigreeFunction
1
0.0 0.5 1.0 1.5 2.0 2.5
Age
1
20 30 40 50 60 70 80
Outcome
1
0.0 0.2 0.4 0.6 0.8 1.0

**Step 4: Remove Outliers Using the Interquartile Range (IQR) Method**

Remove extreme values beyond a reasonable range to improve model robustness.

- IQR = Q3 (75th percentile) – Q1 (25th percentile).
- Values below Q1 - 1.5IQR or above Q3 + 1.5IQR are outliers.
- Calculate lower and upper bounds for each column separately.
- Filter data points to keep only those within bounds.

```
q1, q3 = np.percentile(df['Insulin'], [25, 75])
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
clean_df = df[(df['Insulin'] >= lower) & (df['Insulin'] <=
upper)]
```

**Step 5: Correlation Analysis**

Understand relationships between features and the target variable (Outcome). Correlation helps gauge feature importance.

- **df.corr():** Computes pairwise correlation coefficients between columns.
- Heatmap via seaborn visualizes correlation matrix clearly.
- Sorting correlations with corr['Outcome'].sort_values() highlights features most correlated with the target.

```
corr = df.corr()
plt.figure(dpi=130)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.show()

print(corr['Outcome'].sort_values(ascending=False))
```

**Output:**

| | Outcome |
|---|---|
| Outcome | 1.000000 |
| Glucose | 0.466581 |
| BMI | 0.292695 |
| Age | 0.238356 |
| Pregnancies | 0.221898 |
| DiabetesPedigreeFunction | 0.173844 |
| Insulin | 0.130548 |
| SkinThickness | 0.074752 |
| BloodPressure | 0.065068 |

**dtype:** float64

| | Outcome |
|---|---|
| Outcome | 1.000000 |
| Glucose | 0.466581 |
| BMI | 0.292695 |
| Age | 0.238356 |
| Pregnancies | 0.221898 |
| DiabetesPedigreeFunction | 0.173844 |
| Insulin | 0.130548 |
| SkinThickness | 0.074752 |
| BloodPressure | 0.065068 |

dtype: float64

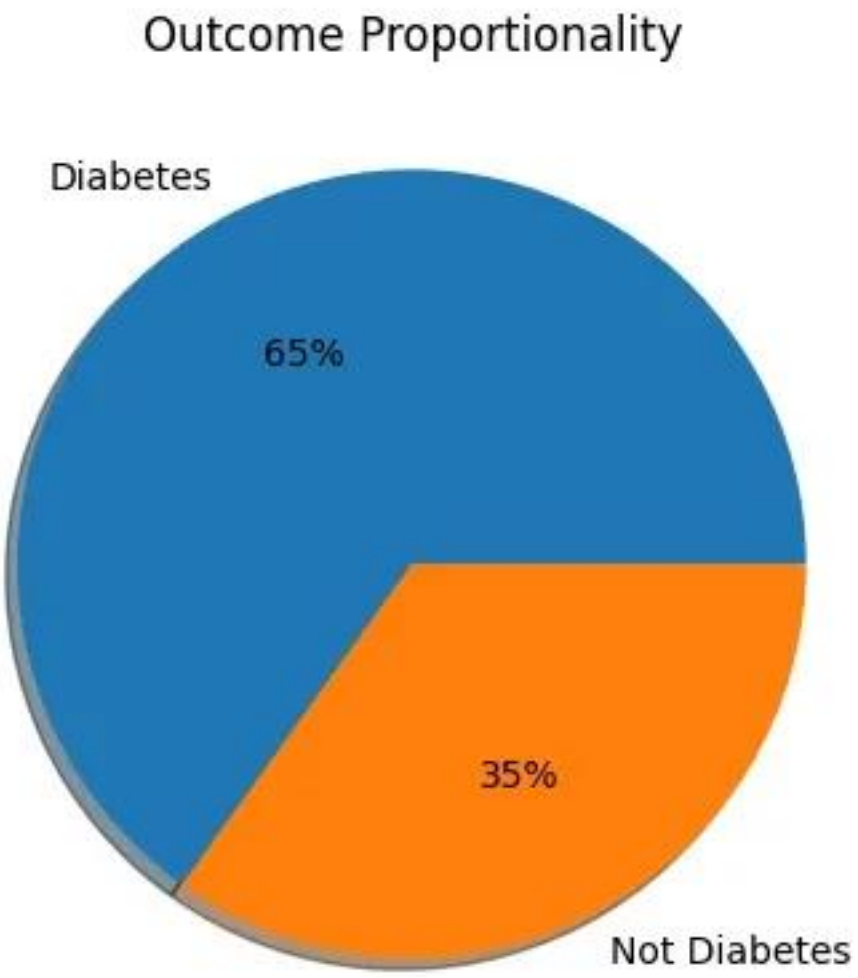
**Step 6: Visualize Target Variable Distribution**

Check if target classes (Diabetes vs Not Diabetes) are balanced, affecting model training and evaluation.

- **plt.pie():** Pie chart to display proportion of each class in the target variable 'Outcome'.

```
plt.pie(df['Outcome'].value_counts(), labels=[
        'Diabetes', 'Not Diabetes'], autopct='%.f%%',
shadow=True)
plt.title('Outcome Proportionality')
plt.show()
```

**Output:**




Result

**Step 7: Separate Features and Target Variable**

Prepare independent variables (features) and dependent variable (target) separately for modeling.

- **df.drop(columns=[...]):** Drops the target column from features.
- Direct column selection df['Outcome'] selects target column.

```
X = df.drop(columns=['Outcome'])
y = df['Outcome']
```

**Step 8: Feature Scaling: Normalization and Standardization**

Scale features to a common range or distribution, important for many ML algorithms sensitive to feature magnitudes.

**1. Normalization (Min-Max Scaling):** Rescales features between 0 and 1. Good for algorithms like k-NN and neural networks.

- **Class:** MinMaxScaler from sklearn.
- **.fit_transform():** Learns min/max from data and applies scaling.

```
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
print(X_normalized[:5])
```

**Output:**

```
array([[0.35294118, 0.74371859, 0.59016393, 0.35353535, 0.        ,
        0.50074516, 0.23441503, 0.48333333],
       [0.05882353, 0.42713568, 0.54098361, 0.29292929, 0.        ,
        0.39642325, 0.11656704, 0.16666667],
       [0.47058824, 0.91959799, 0.52459016, 0.        , 0.        ,
        0.34724292, 0.25362938, 0.18333333],
       [0.05882353, 0.44723618, 0.54098361, 0.23232323, 0.11111111,
        0.41877794, 0.03800171, 0.        ],
       [0.        , 0.68844221, 0.32786885, 0.35353535, 0.19858156,
        0.64232489, 0.94363792, 0.2       ]])
```

**2. Standardization:** Transforms features to have mean = 0 and standard deviation = 1, useful for normally distributed features.

- **Class:** StandardScaler from sklearn.

```
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
print(X_standardized[:5])
```

**Output:**

```
array([[ 0.63994726,  0.84832379,  0.14964075,  0.90726993, -0.69289057,
         0.20401277,  0.46849198,  1.4259954 ],
       [-0.84488505, -1.12339636, -0.16054575,  0.53090156, -0.69289057,
        -0.68442195, -0.36506078, -0.19067191],
       [ 1.23388019,  1.94372388, -0.26394125, -1.28821221, -0.69289057,
        -1.10325546,  0.60439732, -0.10558415],
       [-0.84488505, -0.99820778, -0.16054575,  0.15453319,  0.12330164,
        -0.49404308, -0.92076261, -1.04154944],
       [-1.14185152,  0.5040552 , -1.50468724,  0.90726993,  0.76583594,
         1.4097456 ,  5.4849091 , -0.0204964 ]])
```